Alex Pegoraro

Machine Learning and Control Theory for Accurate and Reliable Temperature Estimation in Power Transformers

Degree Project in Autonomous Systems EIT

IEC Model

Model States:

$\theta_o(t)$: Top-oil Temperature [°C]

$\theta_h(t)$: Hotspot Temperature [°C]

$\theta_h = \theta_o + \theta_{h1} + \theta_{h2}$

$L(\theta_h)$: Loss of Life [min]

Model Inputs:

$K(t)$: Load Factor [p.u.]

$\theta_a(t)$: Ambient Temperature [°C]

$$K_x(t) \stackrel{\text{def}}{=} \left[\frac{1 + K(t)^2 \mathrm{R}}{1 + \mathrm{R}}\right]^x$$

$$K_y(t) \stackrel{\text{def}}{=} K(t)^y$$

$$\theta \stackrel{\text{def}}{=} \begin{bmatrix} \theta_o \\ \theta_{h1} \\ \theta_{h2} \end{bmatrix} \qquad u \stackrel{\text{def}}{=} \begin{bmatrix} \theta_a \\ K_x \\ K_y \end{bmatrix}$$

IEC Model

Differential Formulation:

Employs a set of ODEs

$$\begin{cases} k_{11}\tau_o\theta_o' = -\theta_o + \theta_a + \Delta\theta_{or}K_x \\ \theta_o(0) = \theta_a(0) + \Delta\theta_{or}K_x(0) \end{cases}$$

$$\begin{cases} k_{22}\tau_w\theta_{h1}' = -\theta_{h1} + k_{21}\Delta\theta_{hr}K_y \\ \theta_{h1}(0) = k_{21}\Delta\theta_{hr}K_y(0) \end{cases}$$

$$\begin{cases} \dfrac{\tau_0}{k_{22}}\theta_{h2}' = -\theta_{h2} + \bar{k}_{21}\Delta\theta_{hr}K_y \\ \theta_{h2}(0) = \bar{k}_{21}\Delta\theta_{hr}K_y(0) \end{cases}$$

State Space Representation

$$\begin{cases} \theta'(t) = A\theta(t) + Bu(t) \\ \theta_o(t) = C\theta(t) \\ \theta_h(t) = M\theta(t) \end{cases}$$

$$C = \begin{bmatrix} 1 & 0 & 0 \end{bmatrix}$$

$$M = \begin{bmatrix} 1 & 1 & 1 \end{bmatrix}$$

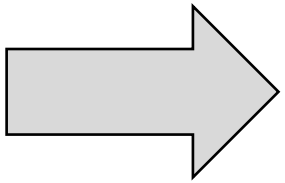
$$A = \begin{bmatrix} -\frac{1}{k_{11}\tau_o} & 0 & 0 \\ 0 & -\frac{1}{k_{22}\tau_w} & 0 \\ 0 & 0 & -\frac{k_{22}}{\tau_0} \end{bmatrix}$$

$$B = \begin{bmatrix} \frac{1}{k_{11}\tau_o} & \frac{\Delta\theta_{or}}{k_{11}\tau_o} & 0 \\ 0 & 0 & k_{21}\frac{\Delta\theta_{hr}}{k_{22}\tau_w} \\ 0 & 0 & \bar{k}_{21}\Delta\theta_{hr}\frac{k_{22}}{\tau_0} \end{bmatrix}$$

Euler Discretization

- Converts a *continuous-time* system into a *discrete-time* system
- Often required to simulate the system

$$\begin{cases} \theta'(t) = A\theta(t) + Bu(t) \\ y(t) = C\theta(t) \end{cases}$$



$$\begin{cases} \theta_i' = A\theta_{i-1} + Bu_i \\ \theta_i = \theta_{i-1} + \Delta t\, \theta_i' \\ y_i = C\theta_i \end{cases}$$

Luenberger Observer

- Exploits the knowledge of an *expected output*
- Introduces a constant gain *L* to be multiplied by the error

$$\begin{cases} \theta_i' = A\theta_{i-1} + Bu_i \\ \theta_i^L = L\left(y_{i-1}^{ref} - y_{i-1}\right) \\ \theta_i = \theta_{i-1} + \Delta t\left(\theta_i' + \theta_i^L\right) \\ y_i = C\theta_i \end{cases}$$

- Approximation of a Kalman Filter:

$$L = QC^T(CQC^T + R)^{-1}$$

Kalman Filter

- Advanced two-stages algorithm
- Propagates *uncertainties* Σ_i, used to compute a variable gain K_i

$$\begin{cases} \theta_i' = A\theta_{i-1} + Bu_i \\ \hat{\theta}_i = \theta_{i-1} + \Delta t\, \theta_i' \\ \hat{y}_i = C\hat{\theta}_i \\ \hat{\Sigma}_i = Q + A\Sigma_{i-1}A^T \end{cases}$$

Predict

$$\begin{cases} K_i = \hat{\Sigma}_i C^T \left(C\hat{\Sigma}_i C^T + R\right)^{-1} \\ \theta_i = \hat{\theta}_i + \Delta t\, K_i \left(y_i^{ref} - \hat{y}_i\right) \\ y_i = C\theta_i \\ \Sigma_i = \hat{\Sigma}_i - K_i C\hat{\Sigma}_i \end{cases}$$

Update

Recurrent Neural Networks

Algorithm 1 RNN

Data: $\theta(0)$, u, A, B, Θ_h^{ref}

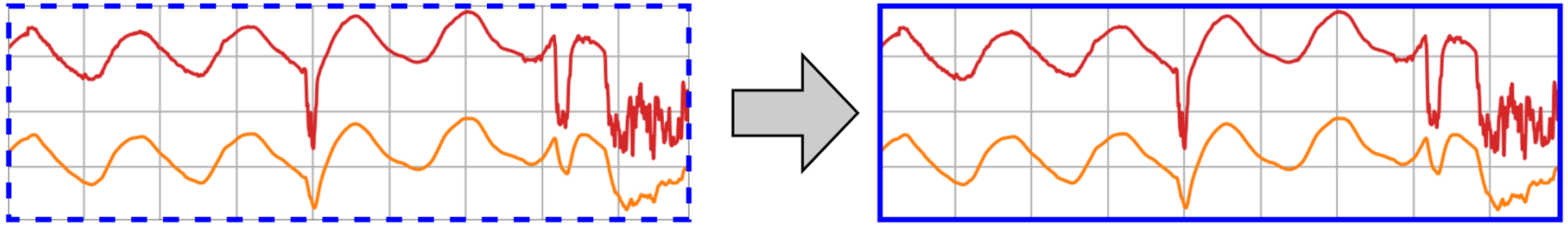
while *Training Loop* **do**

- $A', B' \leftarrow$ `RNN.params()`
- $\Theta \leftarrow$ `Simulate`$(\theta(0), u, A + A', B + B')$
- $\Theta_h \leftarrow M\Theta$
- $\varepsilon \leftarrow$ `Loss`$(\Theta_h, \Theta_h^{\text{ref}})$
- `RNN.backprop`(ε)

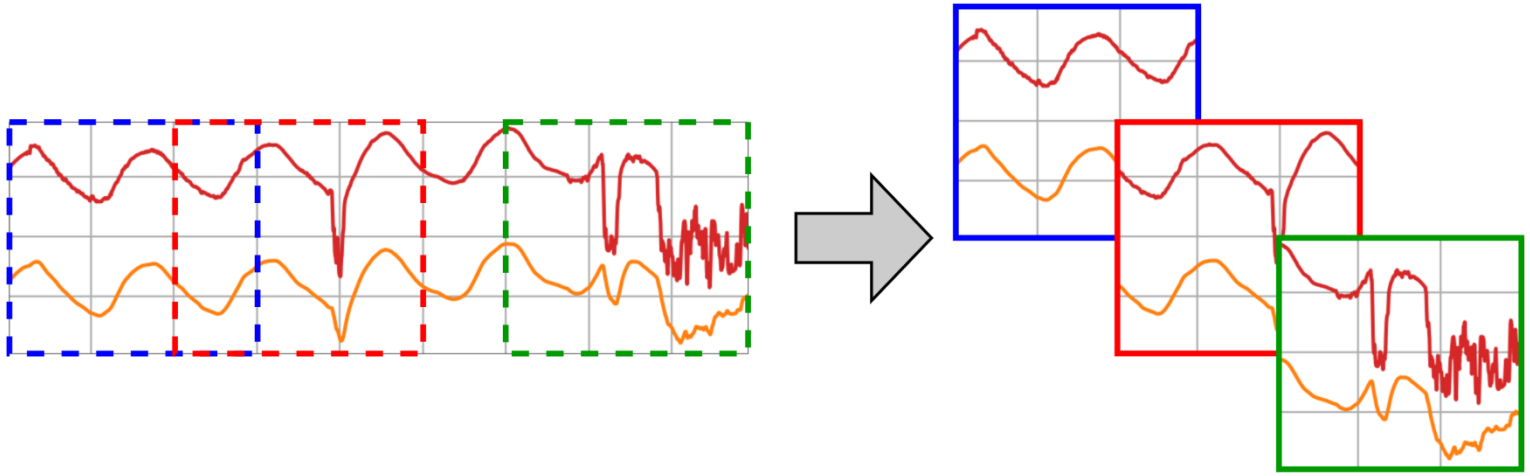
return A', B'

Recurrent Neural Networks

Full-time Training:



Batch Training:



Error Metrics

Mean Absolute Error: $$MAE(x, y) = \frac{1}{N}\sum_{1}^{N} |x_i - y_i|$$

Mean Square Error: $$MSE(x, y) = \frac{1}{N}\sum_{1}^{N} (x_i - y_i)^2$$

Root Mean Square Error: $$RMSE(x, y) = \sqrt{MSE(x, y)}$$

Loss and Regularizers

L_1 Loss: generates sparse entries

$$L_1\left(y, y^{ref}, A', B'\right) = MSE\left(y, y^{ref}\right) + \lambda_A \sum_{i,j} \left|A'_{i,j}\right| + \lambda_B \sum_{i,j} \left|B'_{i,j}\right|$$

L_2 Loss: generates uniform entries

$$L_2\left(y, y^{ref}, A', B'\right) = MSE\left(y, y^{ref}\right) + \lambda_A \sum_{i,j} {A'_{i,j}}^2 + \lambda_B \sum_{i,j} {B'_{i,j}}^2$$

Experiment I
Deterministic Improvements

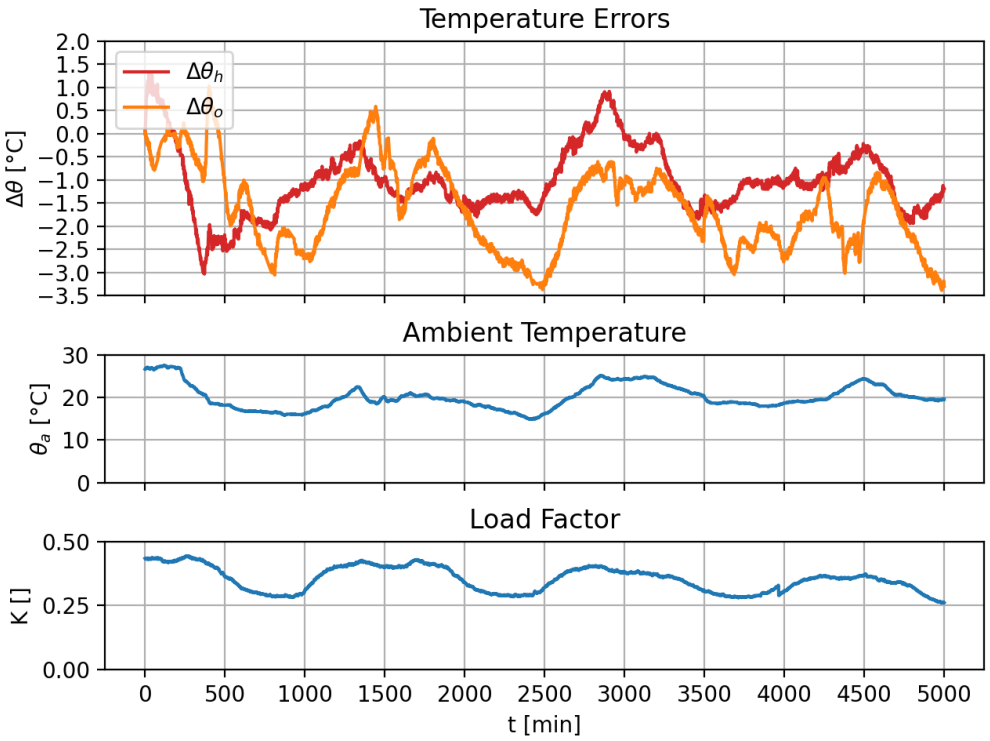
Aim: Compare IEC Model, Luenberger Observer, Kalman Filter on the test dataset

Improvement	**MAE(θ_o)**	**MAE(θ_h)**	**MAE** *tot*	**RMSE(θ_o)**	**RMSE(θ_h)**	**RMSE** *tot*
IEC Model	1.63 °C	1.12 °C	1.38 °C	1.85 °C	1.26 °C	1.58 °C
Luenberger	0.12 °C	0.86 °C	0.49 °C	0.14 °C	1.03 °C	0.74 °C
Kalman	0.10 °C	0.87 °C	0.48 °C	0.12 °C	1.04 °C	0.74 °C

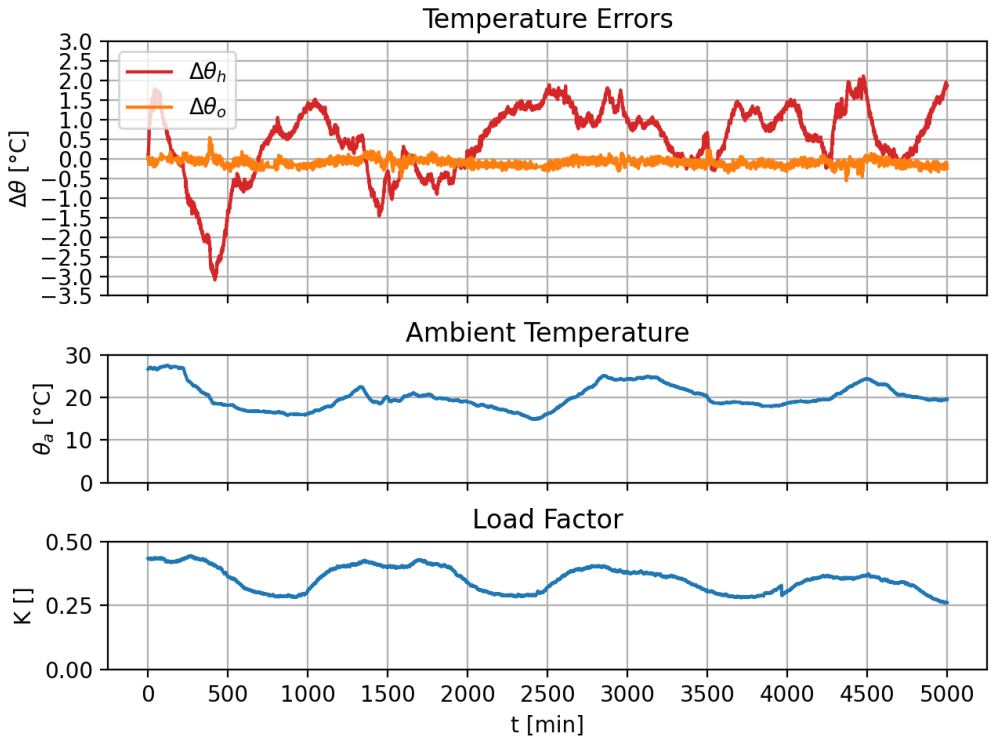
Results:

- Luenberger and Kalman are equivalent
- They reduce mostly θ_o

Experiment I
Deterministic Improvements



IEC Model



Luenberger Observer

Experiment II
RNN Training

Aim: Train the RNN architectures with different constraints and Losses

Results: The proposed matrix modifications

RNN_a (Locked Entries, MSE)

$$A' = \begin{bmatrix} 0 & +2.032e{-3} & +2.032e{-3} \\ 0 & 0 & 0 \\ 0 & 0 & 0 \end{bmatrix}$$

$$B' = 0$$

RNN_b (Locked Entries, MSE)

$$A' = 0$$

$$B' = \begin{bmatrix} +8.470e{-4} & 0 & 0 \\ 0 & 0 & 0 \\ 0 & 0 & 0 \end{bmatrix}$$

Experiment II
RNN Training

`RNN_tot_L1` (Free Entries, L_1) → `RNN_p6` (Locked Entries, MSE)

$$A' = \begin{bmatrix} \boxed{+1.712e{-3}} & \boxed{-1.663e{-3}} & -1.015e{-4} \\ -5.041e{-5} & +7.501e{-4} & \boxed{-1.418e{-3}} \\ -6.884e{-4} & +8.722e{-4} & \boxed{-6.490e{-3}} \end{bmatrix} \quad A' = \begin{bmatrix} +2.503e{-3} & -2.383e{-3} & 0 \\ 0 & 0 & -1.228e{-2} \\ 0 & 0 & -9.042e{-3} \end{bmatrix}$$

$$B' = \begin{bmatrix} \boxed{-1.453e{-3}} & -3.953e{-5} & -4.819e{-5} \\ -7.052e{-5} & -2.063e{-5} & +6.172e{-5} \\ \boxed{-1.960e{-3}} & +2.547e{-5} & -1.378e{-5} \end{bmatrix} \quad B' = \begin{bmatrix} -2.173e{-3} & 0 & 0 \\ 0 & 0 & 0 \\ -4.998e{-3} & 0 & 0 \end{bmatrix}$$

Experiment II
RNN Training

`RNN_tot_L2` (Free Entries, L_2) → `RNN_p5` (Locked Entries, MSE)

$$A' = \begin{bmatrix} \boxed{+1.289e{-3}} & \boxed{-1.056e{-3}} & -5.994e{-4} \\ +3.525e{-4} & +7.384e{-4} & -6.465e{-4} \\ -4.193e{-4} & \boxed{+1.333e{-3}} & \boxed{-1.617e{-3}} \end{bmatrix} \quad A' = \begin{bmatrix} +2.694e{-3} & -1.278e{-3} & 0 \\ 0 & 0 & 0 \\ 0 & -3.721e{-3} & -9.888e{-3} \end{bmatrix}$$

$$B' = \begin{bmatrix} \boxed{-1.266e{-3}} & -3.869e{-5} & -5.597e{-5} \\ -5.597e{-5} & -1.060e{-5} & -7.567e{-6} \\ -8.321e{-4} & -1.378e{-5} & -2.175e{-6} \end{bmatrix} \quad B' = \begin{bmatrix} -3.660e{-3} & 0 & 0 \\ 0 & 0 & 0 \\ 0 & 0 & 0 \end{bmatrix}$$

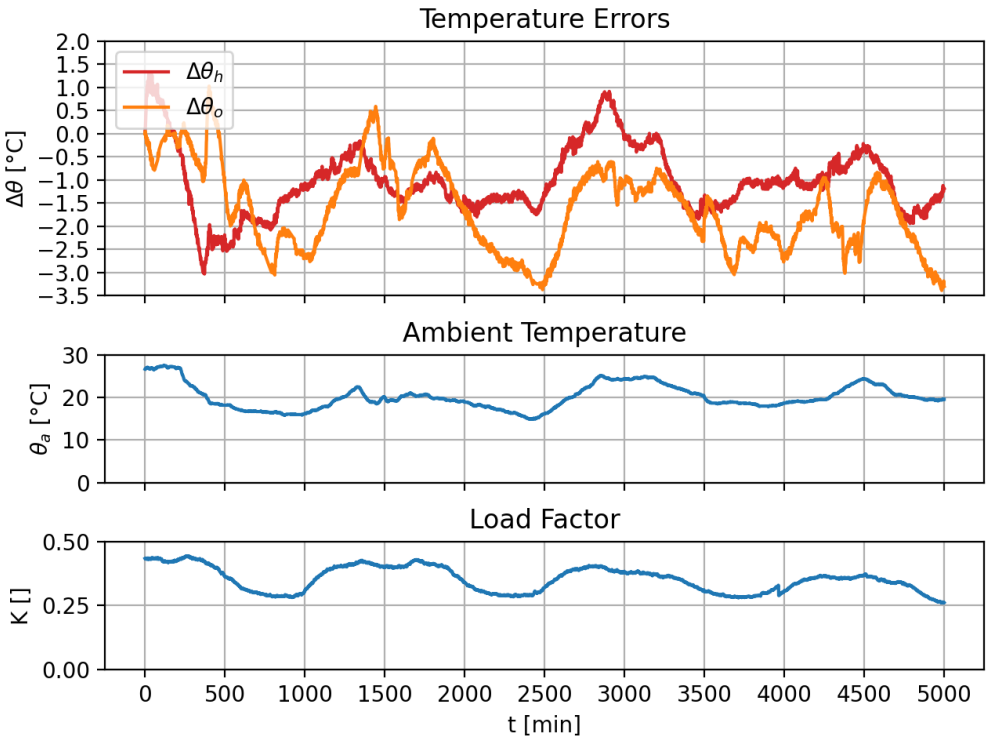
Experiment III
Generalization Capabilities of RNNs

Aim: Compare the RNN architectures on the test dataset

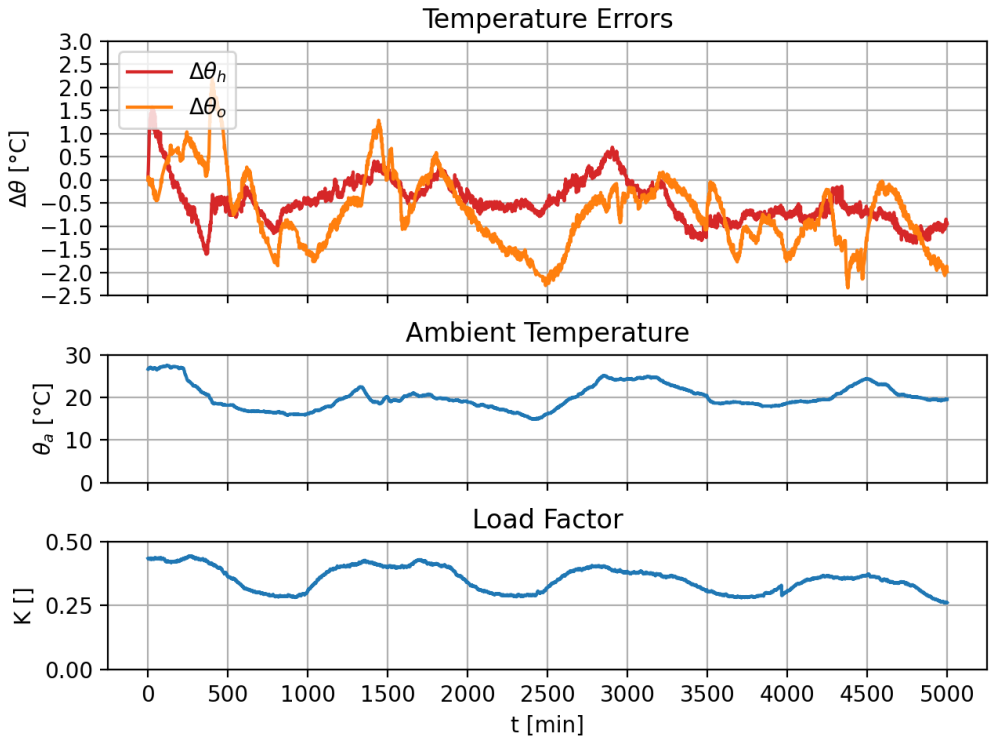
Architecture	MAE	Reduction	RMSE	Reduction
IEC Model	1.38 °C	baseline	1.58 °C	baseline
`RNN_a`	0.83 °C	-39.86 %	1.03 °C	-34.81 %
`RNN_b`	0.75 °C	-45.65 %	0.95 °C	-39.87 %
`RNN_tot_L1`	0.78 °C	-43.48 %	0.96 °C	-39.24 %
`RNN_tot_L2`	0.77 °C	-44.20 %	0.96 °C	-39.24 %
`RNN_p6`	0.72 °C	-47.83 %	0.87 °C	-44.94 %
`RNN_p5`	0.78 °C	-43.48 %	1.06 °C	-32.91 %

Result: The most effective is `RNN_p6`

Experiment III
Generalization Capabilities of RNNs



IEC Model



RNN_p6

Experiment IV
An Hybrid Approach

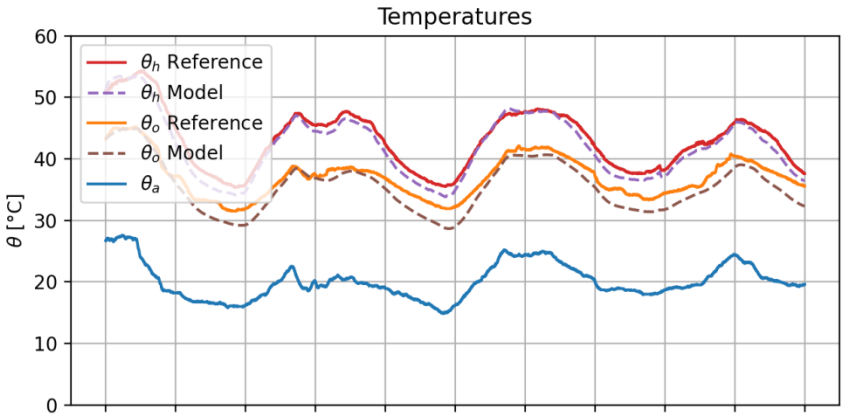
Aim: Test the feasibility of an hybrid approach

Improvement	MAE	Reduction	RMSE	Reduction
IEC Model	1.38 °C	baseline	1.58 °C	baseline
Luenberger	0.49 °C	-64.49 %	0.74 °C	-53.16 %
Kalman	0.48 °C	-65.22 %	0.74 °C	-53.16 %
`RNN_p6`	0.72 °C	-47.83 %	0.87 °C	-44.94 %
`RNN_p6` + Luen.	0.34 °C	-75.36 %	0.54 °C	-65.82 %
`RNN_p6` + Kalman	0.34 °C	-75.36 %	0.54 °C	-65.82 %

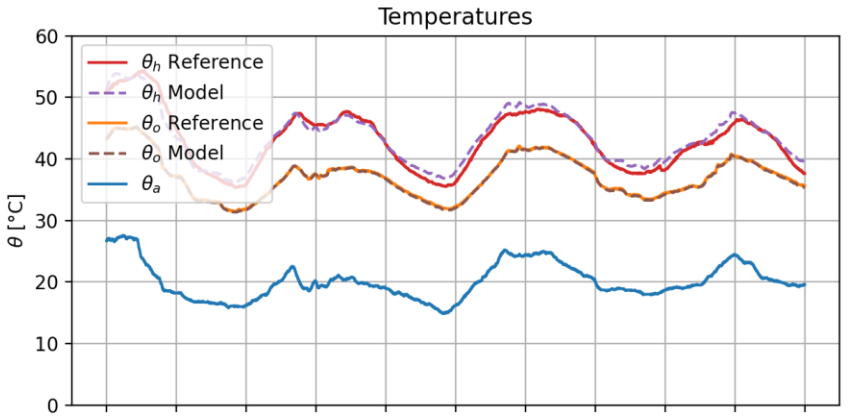
Result: Control Theory improvements and RNNs can be *cumulative*

Experiment IV
An Hybrid Approach

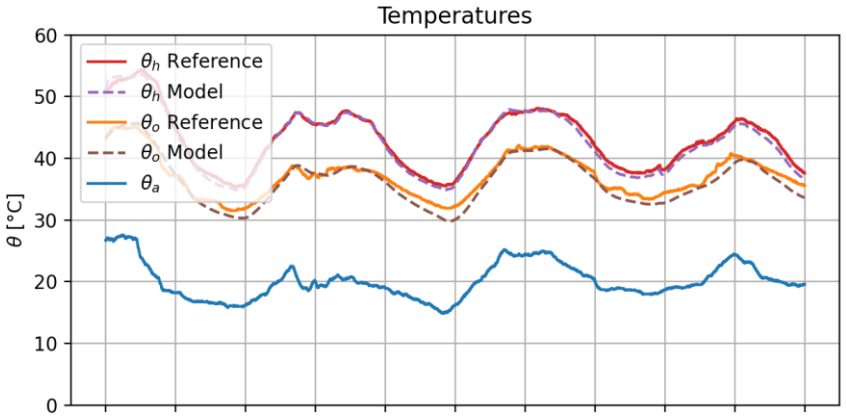
IEC Model



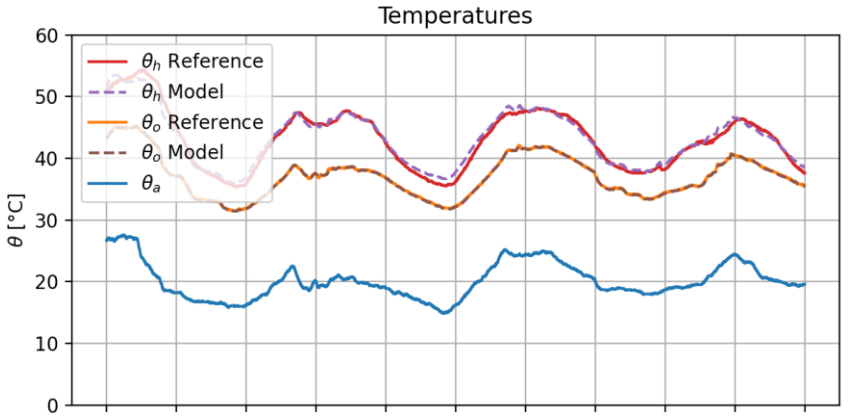
Luenberger



RNN_p6



RNN_p6 + Luenberger



Experiment V
Loss of Life

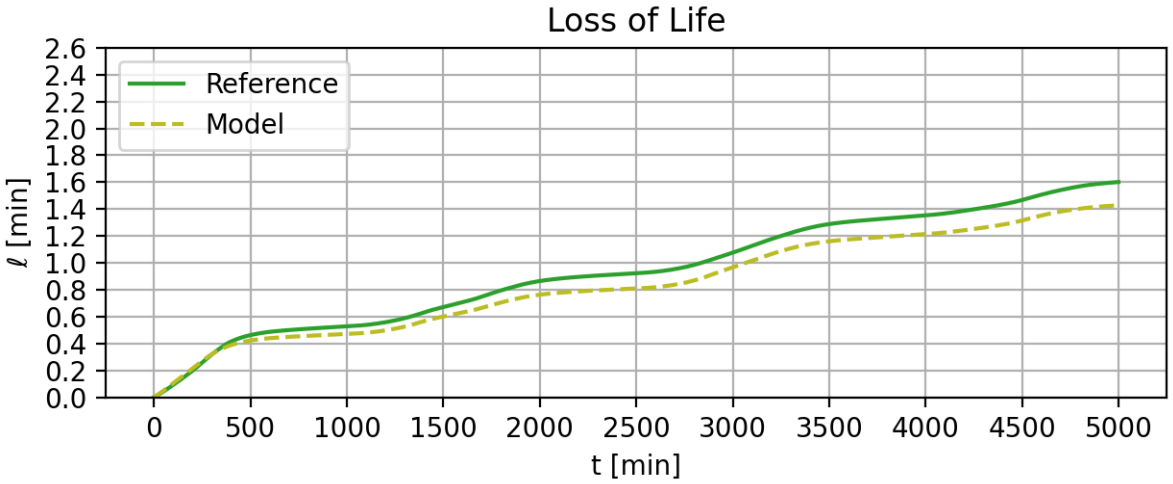
Aim: Check if improvements to θ_h are reflected to L

*Reference Final L is 1.60 min

Improvement	MAE(L)	Reduction	Final ΔL*	Reduction
IEC Model	0.10 min	baseline	-0.17 min	baseline
Luenberger	0.03 min	-70.00 %	+0.06 min	-64.71 %
Kalman	0.03 min	-70.00 %	+0.06 min	-64.71 %
`RNN_p6`	0.03 min	-70.00 %	-0.07 min	-58.82 %
`RNN_p6` + Luen.	0.02 min	-80.00 %	-0.01 min	-94.12 %
`RNN_p6` + Kalman	0.02 min	-80.00 %	-0.01 min	-94.12 %

Result: Improvements to θ_h cause improvements to L

Experiment V
Loss of Life



IEC Model

`RNN_p6` + Luenberger

Future Works

- Simulate the system at *high temperatures*
- Execute the algorithms on a *physical machine*
- Test more precise *discretization techniques*, e.g. Runge Kutta
- Solve the system in *continuous-time*, e.g. Neural ODE
- Test *novel* RNN architecture:
  - Learn A' and B' from the parameters
  - Learn the Luenberger gain
  - Optimize on the Loss of Life

Any Question?

The End